SE Lab 4 — LLM Pair-Programming Chat History

Repository: https://github.com/SETAPESU26/42_marble-tilt-maze
Project: Marble Tilt Maze (Pygame)
Tool used: Claude (claude.ai chat)

MOKSHA

https://github.com/SETAPESU26/42_marble-tilt-maze

[Attached image: Lab 4 VibeCoding instruction sheet — objective: use a vibe coding tool to fix broken code and add features in under three to four attempts; clone the assigned repo, record a 10-second video before changes, fix the code, add the features one by one, record a 10-second video after, keep separate commits per task, do not raise a PR to the main repo; deliverables: videos (before and after), updated code, chat history exported as a doc/pdf.]

CLAUDE

I fetched the repository and read the README. It is a Pygame marble tilt maze in which the marble accelerates toward the mouse cursor. The game is partially broken on purpose, and the README lists four tasks to complete.

Tasks identified from the README

- **Task 1 — Refine Collision Detection:** near a wall's corner the marble bounces off empty space before it visually touches the wall.
- **Task 2 — Implement Game Over Condition:** show a screen with the result (solved with finish time, or timer ran out) instead of printing to the console, then wait for input.
- **Task 3 — Add Replay Option:** after the end screen, let the user play again by choosing Easy, Medium or Hard (tilt strength / friction / time limit), or exit.
- **Task 4 — Add Sound Feedback:** sound effects for bouncing off a wall, reaching the goal, and the timer running out.

Suggested workflow

- Fork or clone the repo into a personal account (no PR to SETAPESU26).
- Record the 10-second "before" video showing the corner-bounce bug.
- Do one task at a time, with a separate commit for each.
- Record the 10-second "after" video, then create a Lab-4 folder with both videos, the code and the exported chat.

MOKSHA

[Uploaded files: 42_marble-tilt-maze-main.zip, Screen_Recording_2026-10-26_120006.mp4]

CLAUDE

I listed and extracted the zip and read every source file (main.py, requirements.txt, game/marble.py, game/wall.py, game/game_engine.py).

Analysis of tasks & bugs in the original code

- **Task 1:** `_resolve_wall_collisions()` used `pygame.Rect.colliderect` between the marble's square bounding box and the wall. Near a corner the square overlaps the wall before the round marble actually touches it, causing a phantom bounce.
- **Task 2:** when the game ended, `render()` only printed to the console. There was no end screen, and the timer kept running after the game was over.
- **Task 3:** there was no way to restart. Difficulty values (tilt, friction, time limit) were hard-coded in `__init__`.
- **Task 4:** there was no audio code at all.

CLAUDE — CODE CHANGES IMPLEMENTED

Work was done in a local git repository with one commit per task:

```
f879827 Baseline: provided starter code
810e2b8 Task 1: true circle-vs-rectangle collision detection
c9d574b Task 2: on-screen game over screen (solved time / timeout)
2453ed0 Task 3: replay with Easy/Medium/Hard difficulty selection or exit
5ab8533 Task 4: sound effects for bounce, goal and timeout
```

1. game/game_engine.py (final version, after all four tasks)

```
import pygame
from .marble import Marble
from .wall import Wall
from .sound import SoundManager

# Game Engine

WHITE = (255, 255, 255)
DARK = (40, 40, 50)
WALL_COLOR = (90, 90, 110)
GOAL_COLOR = (60, 200, 120)

# Difficulty presets: tilt strength, friction, time limit (ms)
DIFFICULTIES = {
    "Easy":   {"tilt_strength": 0.5, "friction": 0.04,  "time_limit_ms": 60000},
    "Medium": {"tilt_strength": 0.6, "friction": 0.02,  "time_limit_ms": 45000},
    "Hard":   {"tilt_strength": 0.8, "friction": 0.008, "time_limit_ms": 30000},
}
DIFFICULTY_KEYS = {pygame.K_1: "Easy", pygame.K_2: "Medium", pygame.K_3: "Hard"}


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.max_speed = 9
        self.walls = self._build_maze()
        self.goal_x, self.goal_y, self.goal_radius = width - 60, height - 60, 22

        self.font = pygame.font.SysFont("Arial", 26)
        self.big_font = pygame.font.SysFont("Arial", 54, bold=True)
        self.quit_requested = False
        self.sound = SoundManager()

        self.start_round("Medium")

    def start_round(self, difficulty):
        """(Re)start a round with the chosen difficulty preset."""
        self.difficulty = difficulty
        settings = DIFFICULTIES[difficulty]
        self.tilt_strength = settings["tilt_strength"]
        self.friction = settings["friction"]
        self.time_limit_ms = settings["time_limit_ms"]

        self.marble = Marble(50, 50)
        self.start_ticks = pygame.time.get_ticks()
        self.game_over = False
        self.result = None  # "solved" or "timeout"
        self.finish_time_ms = None
        self.end_ticks = None

    def _build_maze(self):
        walls = []
        t = 16  # wall thickness

        # outer boundary
        walls.append(Wall(0, 0, self.width, t))
        walls.append(Wall(0, self.height - t, self.width, t))
        walls.append(Wall(0, 0, t, self.height))
        walls.append(Wall(self.width - t, 0, t, self.height))

        # a few internal walls forming a simple winding path
        walls.append(Wall(0, 140, self.width - 140, t))
        walls.append(Wall(140, 260, self.width - 140, t))
        walls.append(Wall(0, 380, self.width - 140, t))

        return walls
```

```
def handle_event(self, event):
    # Gameplay is driven by the mouse position (handle_input); events
    # are only needed for the end screen.
    if self.game_over and event.type == pygame.KEYDOWN:
        if event.key in DIFFICULTY_KEYS:
            self.start_round(DIFFICULTY_KEYS[event.key])
        elif event.key in (pygame.K_ESCAPE, pygame.K_q):
            self.quit_requested = True

def handle_input(self):
    if self.game_over:
        return

    mouse_x, mouse_y = pygame.mouse.get_pos()
    dx = mouse_x - self.width // 2
    dy = mouse_y - self.height // 2
    dist = max(1, (dx ** 2 + dy ** 2) ** 0.5)
    ax = (dx / dist) * self.tilt_strength
    ay = (dy / dist) * self.tilt_strength
    self.marble.vx += ax
    self.marble.vy += ay

def update(self):
    if self.game_over:
        return

    elapsed = pygame.time.get_ticks() - self.start_ticks
    if elapsed >= self.time_limit_ms:
        self.game_over = True
        self.result = "timeout"
        self.end_ticks = pygame.time.get_ticks()
        self.sound.play_timeout()
        return

    self.marble.vx *= (1 - self.friction)
    self.marble.vy *= (1 - self.friction)

    speed = (self.marble.vx ** 2 + self.marble.vy ** 2) ** 0.5
    if speed > self.max_speed:
        scale = self.max_speed / speed
        self.marble.vx *= scale
        self.marble.vy *= scale

    self.marble.x += self.marble.vx
    self.marble.y += self.marble.vy

    self._resolve_wall_collisions()

    gx = self.goal_x - self.marble.x
    gy = self.goal_y - self.marble.y
    if (gx ** 2 + gy ** 2) ** 0.5 <= self.goal_radius:
        self.game_over = True
        self.result = "solved"
        self.finish_time_ms = elapsed
        self.end_ticks = pygame.time.get_ticks()
        self.sound.play_goal()

def _resolve_wall_collisions(self):
    """True circle-vs-rectangle collision.

    For each wall, find the point on the rectangle closest to the
    marble's centre. The marble only touches the wall if that point
    is closer than the marble's radius, so corners no longer cause
    phantom bounces off empty space.
    """
    m = self.marble
    for wall in self.walls:
        rect = wall.rect()

        # Closest point on the wall rectangle to the marble centre
        cx = max(rect.left, min(m.x, rect.right))
        cy = max(rect.top, min(m.y, rect.bottom))

        dx = m.x - cx
        dy = m.y - cy
        dist_sq = dx * dx + dy * dy

        if dist_sq >= m.radius * m.radius:
            continue  # not touching
```

```
            if dist_sq > 0:
                # Centre is outside the wall: push out along the contact normal
                dist = dist_sq ** 0.5
                nx, ny = dx / dist, dy / dist
                penetration = m.radius - dist
            else:
                # Centre is inside the wall: push out along the shallowest axis
                left = m.x - rect.left
                right = rect.right - m.x
                top = m.y - rect.top
                bottom = rect.bottom - m.y
                smallest = min(left, right, top, bottom)
                if smallest == left:
                    nx, ny, penetration = -1, 0, left + m.radius
                elif smallest == right:
                    nx, ny, penetration = 1, 0, right + m.radius
                elif smallest == top:
                    nx, ny, penetration = 0, -1, top + m.radius
                else:
                    nx, ny, penetration = 0, 1, bottom + m.radius

            # Separate the marble from the wall
            m.x += nx * penetration
            m.y += ny * penetration

            # Bounce: only reflect the velocity component moving into the wall
            vn = m.vx * nx + m.vy * ny
            if vn < 0:
                self.sound.play_bounce(-vn)
                m.vx -= (1 + 0.3) * vn * nx
                m.vy -= (1 + 0.3) * vn * ny

    def render(self, screen):
        screen.fill(DARK)

        for wall in self.walls:
            pygame.draw.rect(screen, WALL_COLOR, wall.rect())

        pygame.draw.circle(screen, GOAL_COLOR, (self.goal_x, self.goal_y), self.goal_radius)
        pygame.draw.circle(screen, WHITE, (int(self.marble.x), int(self.marble.y)), self.marble.radius)

        now = self.end_ticks if self.game_over else pygame.time.get_ticks()
        elapsed = now - self.start_ticks
        seconds_left = max(0, (self.time_limit_ms - elapsed) // 1000)
        timer_text = self.font.render(f"Time: {seconds_left}s", True, WHITE)
        screen.blit(timer_text, (10, 10))
        level_text = self.font.render(self.difficulty, True, WHITE)
        screen.blit(level_text, (self.width - level_text.get_width() - 24, 20))

        if self.game_over:
            self._render_end_screen(screen)

    def _render_end_screen(self, screen):
        overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
        overlay.fill((0, 0, 0, 170))
        screen.blit(overlay, (0, 0))

        if self.result == "solved":
            title, color = "Maze Solved!", GOAL_COLOR
            detail = f"Finish time: {self.finish_time_ms / 1000:.1f}s"
        else:
            title, color = "Time's Up!", (230, 90, 90)
            detail = "The maze was not solved in time."

        lines = [
            (self.big_font, title, color, -60),
            (self.font, detail, WHITE, 0),
            (self.font, "Play again:  1 Easy   2 Medium   3 Hard", WHITE, 60),
            (self.font, "Press Esc or Q to exit", WHITE, 100),
        ]
        for font, text, col, offset in lines:
            surf = font.render(text, True, col)
            rect = surf.get_rect(center=(self.width // 2, self.height // 2 + offset))
            screen.blit(surf, rect)
```

2. game/sound.py (new file, Task 4)

```
import array
import math
```

```
import pygame

SAMPLE_RATE = 22050


def _tone(notes, volume=0.4):
    """Build a mono 16-bit sound from [(frequency_hz, seconds), ...]."""
    samples = array.array("h")
    for freq, seconds in notes:
        n = int(SAMPLE_RATE * seconds)
        for i in range(n):
            envelope = 1.0 - i / n  # fade out so notes don't click
            value = math.sin(2 * math.pi * freq * i / SAMPLE_RATE)
            samples.append(int(value * envelope * volume * 32767))
    return pygame.mixer.Sound(buffer=samples.tobytes())


class SoundManager:
    """Generates and plays simple sound effects (no audio files needed).

    If audio cannot be initialised (no sound device), every method becomes
    a silent no-op so the game still runs.
    """

    BOUNCE_COOLDOWN_MS = 80
    MIN_BOUNCE_SPEED = 1.0

    def __init__(self):
        self.enabled = False
        self._last_bounce = 0
        try:
            if not pygame.mixer.get_init():
                pygame.mixer.init(frequency=SAMPLE_RATE, size=-16, channels=1)
            self.bounce = _tone([(220, 0.07)], volume=0.35)
            self.goal = _tone([(523, 0.12), (659, 0.12), (784, 0.25)])
            self.timeout = _tone([(392, 0.2), (330, 0.2), (262, 0.4)])
            self.enabled = True
        except (pygame.error, NotImplementedError):
            pass

    def play_bounce(self, impact_speed):
        if not self.enabled or impact_speed < self.MIN_BOUNCE_SPEED:
            return
        now = pygame.time.get_ticks()
        if now - self._last_bounce < self.BOUNCE_COOLDOWN_MS:
            return
        self._last_bounce = now
        self.bounce.set_volume(min(1.0, impact_speed / 6))
        self.bounce.play()

    def play_goal(self):
        if self.enabled:
            self.goal.play()

    def play_timeout(self):
        if self.enabled:
            self.timeout.play()
```

3. main.py (small change, Task 2: exit when the engine requests it)

```
import pygame
from game.game_engine import GameEngine

# Initialize pygame/Start application
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 600, 500
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Marble Tilt Maze - Pygame Version")

# Clock
clock = pygame.time.Clock()
FPS = 60

# Game loop
engine = GameEngine(WIDTH, HEIGHT)

def main():
    running = True
```

```
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            engine.handle_event(event)

        if engine.quit_requested:
            running = False

        engine.handle_input()
        engine.update()
        engine.render(SCREEN)

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()

if __name__ == "__main__":
    main()
```

Testing performed

Each task was tested headlessly (SDL dummy video/audio drivers) with small scripts:

- **Task 1:** a marble placed diagonally near a wall corner (outside the true circle distance) was not moved; a marble genuinely touching the wall was pushed out to y = 168 and its velocity was reflected.
- **Task 2:** forcing a timeout and placing the marble on the goal both set `game_over` with the right result, and the end-screen overlay rendered correctly.
- **Task 3:** keys 1/2/3 restarted the round with the matching preset (e.g. Hard: tilt 0.8, friction 0.008, 30 s); Esc set `quit_requested`.
- **Task 4:** the bounce, goal and timeout sound methods were each triggered at the right moment, and a 600-frame simulation ran without errors.

CLAUDE — FINAL RESPONSE

All four tasks were implemented and tested headlessly. Sound and real mouse play were not tested in the chat environment, so the game should be run once with `python main.py` before recording the "after" video.

Summary of completed tasks

- **Task 1 — Collision:** the square-box overlap check was replaced by a circle-vs-rectangle test using the closest point on each wall. A marble near a corner no longer bounces off empty space, while real contact still pushes it out and bounces it at 30% speed.
- **Task 2 — End screen:** a dark overlay shows "Maze Solved!" with the finish time, or "Time's Up!", instead of printing to the console. The timer freezes when the round ends.
- **Task 3 — Replay:** the end screen offers 1 Easy, 2 Medium or 3 Hard to play again, and Esc or Q to exit. Easy: tilt 0.5, friction 0.04, 60 s. Medium: tilt 0.6, friction 0.02, 45 s (original). Hard: tilt 0.8, friction 0.008, 30 s. The current difficulty is shown in the top-right corner.
- **Task 4 — Sound:** effects for wall bounce (volume scales with impact), reaching the goal, and the timer running out. They are generated in code, so no audio files or extra dependencies are needed. If audio cannot start, the game runs silently.

Deliverables produced: marble-tilt-maze-updated.zip (updated project) and task1.patch to task4.patch (one change per task, for separate commits).

MOKSHA

what are these patch files and where should i impliment them

CLAUDE

A patch file is a text file that lists the exact lines of code to add or remove. Each of the four patches is the diff for one task, so applying it makes that change to your code, and you can commit each task separately as the lab requires. Apply them in your own copy of the repo (your fork or clone), not the original SETAPESU26 repo.

```
git apply ../patches/task1.patch
git add -A
git commit -m "Task 1: circle-vs-rect collision"

# repeat in order for task2.patch, task3.patch and task4.patch

python main.py
git push
```

Alternative: unzip marble-tilt-maze-updated.zip and copy main.py and the game/ folder over the repo's files, then commit. The lab asks for a separate commit per task, so the patches are the cleaner route.